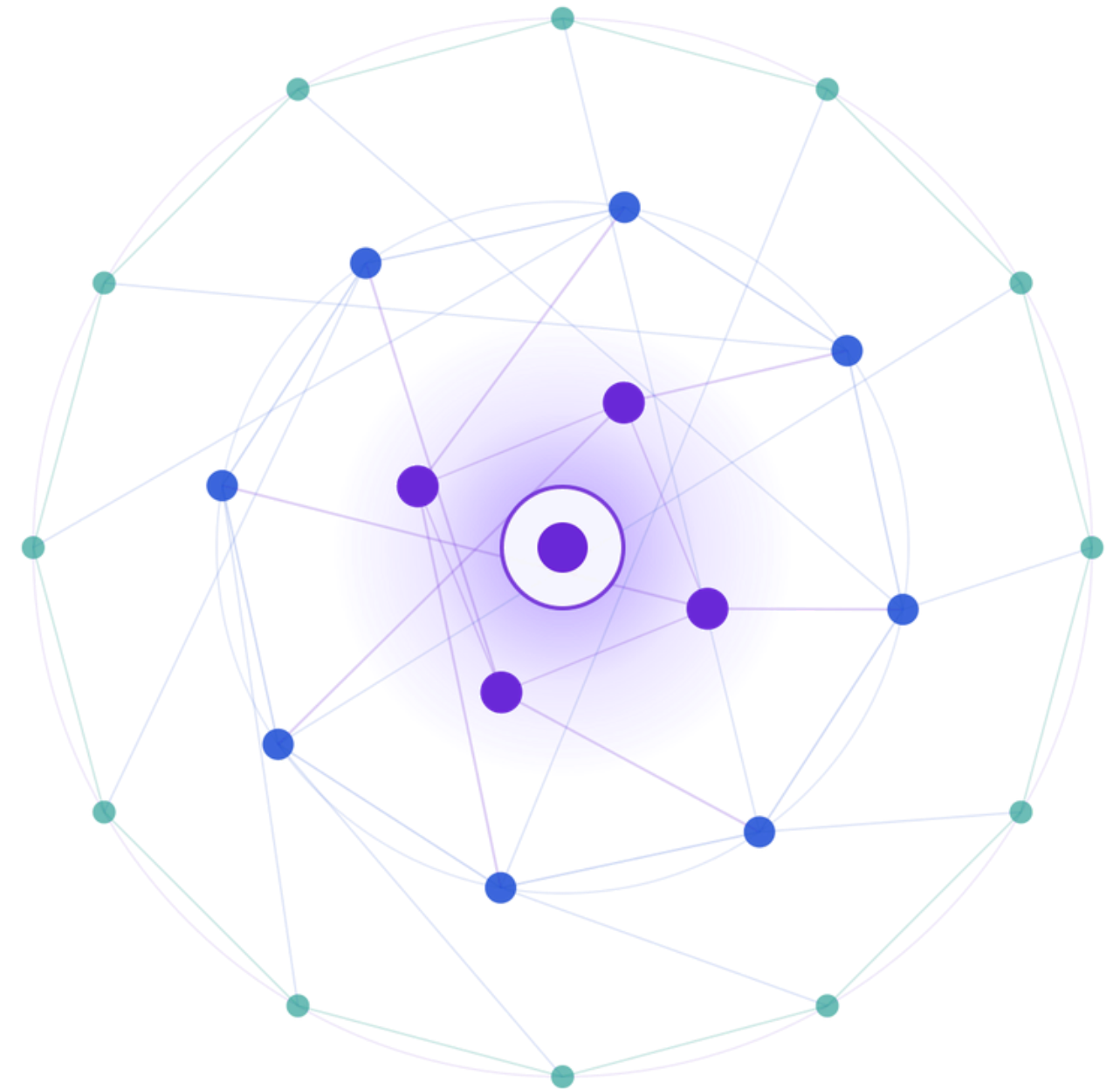


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Theme 1 - Agentic Code Intelligence
- Team Name - CodeLens
- College Name - SRMIST, KTR
- Member Name & Email 1- Rithanya M (rm4150@srmist.edu.in)
- Member Name & Email 2- Rithika R (rr5378@srmist.edu.in)
- Submission Github link - <https://github.com/R1thanya/codelens-retrieval>



Theme

- Natural-language code retrieval ranks code blocks by relevance to a question.
- At repository scale, locating the right function is the bottleneck; sending the full corpus to an LLM is too slow and too large.
- Scope: retrieval only. CodeLens returns ranked snippets and locations, not generated explanations.
- Example query: Which function trims the raw input string?

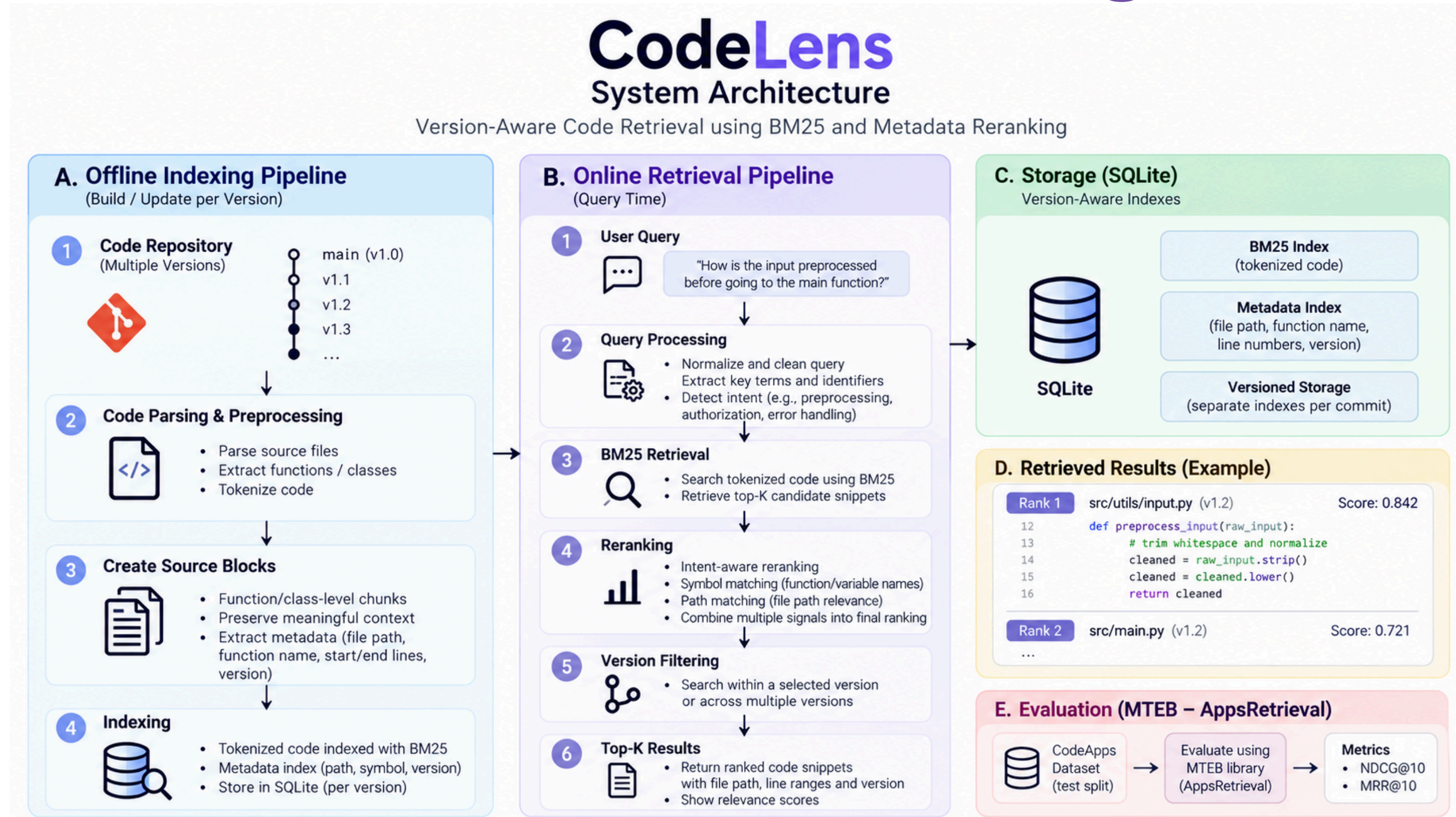
Existing Solutions & Gaps

- Pure lexical retrieval misses paraphrases and behavior described with vocabulary absent from code.
- Dense semantic retrieval handles paraphrases but can miss exact symbols and identifiers.
- Whole-file matching dilutes function-level evidence; fixed chunks can cut through useful context.
- Changing code needs revision-specific indexes without wasting time re-embedding unchanged blocks.

Our Solutions & Architecture Diagram

- Code-aware preprocessing — tokenizes code and preserves meaningful source blocks.
- Version-aware indexing — maintains searchable indexes across Git revisions.
- BM25 lexical retrieval — efficiently matches natural-language queries with code.
- Intent-aware reranking — improves retrieval using query intent and behavioral signals.
- Symbol & path matching — uses function names, identifiers and file paths as explicit signals.
- Version-aware search — supports selected-version and cross-version retrieval.
- CPU-efficient design — runs locally without GPU or API dependencies.
- Top-K ranked results — returns source snippets with paths and line ranges.

Our Solutions & Architecture Diagram



Demo & Product Walkthrough

- Index a Git repository or source folder.
- Ask a natural-language code question.
- Retrieve relevant snippets using BM25 + intent/path/symbol reranking.
- Rank results and inspect file paths, line ranges and scores.
- Compare results across indexed versions.

Tools and tech stack used

- Demo: Python 3.10+, SQLite, standard-library HTTP, BM25 + intent/path/symbol reranking.
- Demo + benchmark: CodeLens BM25 (sparse lexical retrieval), implemented in Python.
- Signals: tokenized code and query, BM25, query intent, identifiers, symbol/path matches.
- Benchmark adapter uses the same BM25/query-intent ranker; MTEB 2.21.8, CPU.
- No learned embedding model, GPU, API key, or answer generation is used.

Impact & Use case

- Find input cleanup, authorization checks, error paths, cache invalidation and call chains without reading every file.
- Reviewers can inspect exact matching blocks and source locations rather than generated summaries.
- Version-pinned search supports commit review and regression investigation.
- Expected use: developer navigation in a changing codebase.
Productivity impact remains to be measured.

Innovation highlights, results and limitations

- MTEB AppsRetrieval test | CodeLens BM25 | NDCG@10: 0.01645 | MRR@10: 0.01333.
- One CPU run; MTEB 2.21.8. Native appsretrieval_results.json is included.
- A measured baseline, not a winning-score claim; retrieval quality needs tuning.
- Limits: heuristic chunks; no controlled speed comparison or significance study.

What's next

- Tune with validation data, freeze configuration, then rerun the official test benchmark.
- Improve intent matching and code-aware chunking; compare each change on held-out data.
- Add syntax-tree chunking and incremental file-level rebuilds for larger repositories.
- Measure index build time and p50/p95 query latency on a stated CPU and repository.

Brownie points slide(differentiation)

- Retrieval stays inspectable: every result is a source block with a path and line range.
- Version-scoped SQLite indexes can be rebuilt independently for different commits.
- Local BM25 ranks with explicit lexical, intent, symbol and path signals.
- Evolutionary search is available across indexed revisions; robust function lineage and its quality are not yet evaluated.

Checklist- Updated on Public GitHub

- Working prototype code — Public/Shared GitHub Repo (Y/N): Y
- README with reproducible setup instructions (Y/N): Y
- Demo video, max 5 minutes (YouTube/Drive): Drive Link
- [https://drive.google.com/drive/folders/13MAI3jJqf_vgIKZyBtklfmaL
Gsd73E_9?usp=sharing](https://drive.google.com/drive/folders/13MAI3jJqf_vgIKZyBtklfmaLGsd73E_9?usp=sharing)
- Presentation file (PPT/PDF) (Y/N): PDF — Y

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India